

**AICTE - EDUSKILLS VIRTUAL INTERNSHIP
PROJECT REPORT**

"Simple Finite State Machine (FSM) in Java"

SUBMITTED BY:

Student Name: Kongala Sruthi

Program: B.Tech

Branch: CSE

Semester: Unknown

Academic Year: None-2027

SUBMITTED TO:

Sree Venkateswara College of Engineering

Internship Details:

AICTE Student ID: STU688b79ad5356c1753971117

APAAR ID: 591818067474

Internship Duration: 10 Weeks (Jan 2026 - Mar 2026)

Supported by: Eduskills Academy

Under the Guidance of

Project Manager

Eduskills

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Mr./Ms. Kongala Sruthi**, a student of **Sree Venkateswara College of Engineering**, has successfully completed **10-Week (Jan 2026 - Mar 2026)** AICTE – EduSkills Virtual Internship Program.

During the internship, he/she worked on the project titled "**Simple Finite State Machine (FSM) in Java**". His/her performance during the internship was found to be satisfactory, and he/she demonstrated good technical and professional skills.

We wish him/her success in future endeavors.

Sagar Bala Behera

Mentor, EduSkills Academy

Date: 08-03-2026

DECLARATION

I hereby declare that the work presented in this Online Virtual Internship Project Report is my original work carried out under the guidance of my industry mentor and faculty guide. This work has not been submitted earlier for the award of any degree or diploma.

Kongala Sruthi

(Signature)

Date: 08-03-2026

ACKNOWLEDGEMENT

I express my sincere gratitude to **AICTE & EduSkills** for providing me with the opportunity to undertake this online virtual internship. I would like to thank my industry mentor **Mr./Ms. Sagar Bala Behera** for his/her valuable guidance, continuous support, and technical insights throughout the internship period.

I am also thankful to the Department of **CSE** for their encouragement and support.

ABSTRACT

This project details the design and implementation of a Finite State Machine (FSM) in Java. The primary goal is to demonstrate the core concepts of FSMs, including state management, event-driven transitions, and maintaining state history. The project successfully builds functional FSM models for various scenarios, showcasing their versatility and importance in software design.

Table of Contents

1. Introduction	2
2. Organization Profile	3
3. Internship Overview	4
4. Problem Statement	5
5. Methodology	6
6. Tools & Technologies	7
7. Implementation	8
8. Learning Outcomes	11
9. Challenges & Solutions	12
10. Conclusion	13
11. Future Scope	14
12. References	15
13. Annexure	16

1. Introduction

Industry-oriented learning through internships plays a crucial role in bridging the gap between academic knowledge and real-world applications. This online virtual internship provided hands-on exposure to **Java Full Stack With Project** and its application in modern software solutions.

2. Organization Profile

Organization Name: EduSkills Academy

Domain: EdTech & Skill Development

Services: Virtual Internships

Vision: To empower students with industry-relevant skills.

3. Internship Overview

- Internship Mode: **Online / Virtual**
- Duration: **10 Weeks**
- Platform Used: **EduSkills LMS**
- Role: **Intern**

4. Problem Statement & Objectives

Problem Statement:

This project presents a comprehensive implementation of a Finite State Machine (FSM) using Java. It explores the theoretical underpinnings of FSMs and translates them into practical applications through a series of distinct problem implementations. The project covers essential FSM functionalities, including state definition, transition logic based on input events, and the tracking of state changes. The objective is to provide a clear and functional demonstration of how FSMs can be utilized to model and manage complex behaviors in software systems, ranging from simple toggles to more intricate state-dependent operations.

Objectives:

- To design and implement a solution for the given problem statement.
- To utilize modern tools and algorithms.
- To optimize performance and code quality.

5. Project Description & Methodology

The development of the Finite State Machine (FSM) was approached systematically. Initially, the fundamental FSM concept was established, defining states and transitions triggered by events. This was followed by implementing distinct FSM models for practical problems: a basic toggle for a 'Light Switch,' command-based transitions for a 'Secure Gate,' a cyclical state progression for a 'Traffic Light,' tracking state visits for a 'Path History' feature, and incorporating error handling for 'Guarded Transitions.' Each implementation focused on clear state representation and accurate transition logic.

6. Tools & Technologies Used

Programming Language: Java

Core Concepts: Finite State Machines (FSM), State Management, Event Handling, Object-Oriented Programming (OOP), Class Design.

7. Implementation & Results

Problem 1: Light Switch: Basic Toggle

Description: Implement a Finite State Machine that represents a simple light switch. The FSM should transition between two states, 'OFF' and 'ON', toggling its current state upon receiving a 'TOGGLE' input event. This problem aims to demonstrate basic state transitions and input handling.

```
import java.util.*;

public class Solution {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        List<String> results = new ArrayList<>();
        String state = "OFF";

        while (scanner.hasNextLine()) {
            String line = scanner.nextLine().trim();

            if (line.equals("TOGGLE")) {
                if (state.equals("OFF")) {
                    state = "ON";
                } else {
                    state = "OFF";
                }

                results.add("{\"key\": \"" + state + "\"}");
            }
        }

        scanner.close();

        System.out.print("[");
        for (int i = 0; i < results.size(); i++) {
            System.out.print(results.get(i));
            if (i < results.size() - 1) {
                System.out.print(",");
            }
        }
        System.out.print("]");
    }
}
```

Problem 2: Secure Gate: Specific Commands

Description: Develop a Finite State Machine for a secure gate system. The FSM should manage 'LOCKED' and 'OPEN' states. Specific input commands ('UNLOCK', 'LOCK') should trigger transitions between these states, ensuring that the gate operates according to defined access protocols.

```
import java.util.*;

public class Solution {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        List<String> results = new ArrayList<>();
        String state = "LOCKED";

        while (scanner.hasNextLine()) {
            String line = scanner.nextLine().trim();

            if (line.isEmpty()) {
                results.add("{\"key\":[]}");
            } else {
                if (line.equals("UNLOCK")) {
                    state = "OPEN";
                } else if (line.equals("LOCK")) {
                    state = "LOCKED";
                }

                results.add("{\"key\":[" + state + "]}");
            }
        }

        scanner.close();

        System.out.print("[");
        for (int i = 0; i < results.size(); i++) {
            System.out.print(results.get(i));
            if (i < results.size() - 1) System.out.print(",");
        }
        System.out.println("]");
    }
}
```

Problem 3: Traffic Light: State Cycles

Description: Design a Finite State Machine that simulates a traffic light cycle. The FSM should progress through a sequence of states: 'GREEN', 'YELLOW', and 'RED', returning to 'GREEN' after 'RED'. Each 'NEXT' command should advance the FSM to its subsequent state in the defined cycle.

```

import java.util.*;

public class Solution {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        List<String> results = new ArrayList<>();
        String state = "RED";

        while (scanner.hasNextLine()) {
            String line = scanner.nextLine().trim();

            if (line.isEmpty()) {
                results.add("{\"key\":[ ]}");
            } else {
                if (line.equals("NEXT")) {
                    if (state.equals("RED")) {
                        state = "GREEN";
                    } else if (state.equals("GREEN")) {
                        state = "YELLOW";
                    } else if (state.equals("YELLOW")) {
                        state = "RED";
                    }
                }

                results.add("{\"key\":[\"" + state + "\"]}");
            }
        }

        scanner.close();

        System.out.print("[");
        for (int i = 0; i < results.size(); i++) {
            System.out.print(results.get(i));
            if (i < results.size() - 1) System.out.print(",");
        }
        System.out.println("]");
    }
}

```

Problem 4: State History Tracker

Description: Implement a Finite State Machine that records a history of all visited states. The FSM begins in a 'START' state. Each transition to a new state should be appended to a log, demonstrating the complete path taken from the initial state through a series of inputs. This problem focuses on state tracking and history maintenance.

```
import java.util.*;

public class Solution {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        List<String> results = new ArrayList<>();
        StringBuilder history = new StringBuilder("START");

        while (scanner.hasNextLine()) {
            String line = scanner.nextLine().trim();

            if (!line.isEmpty()) {
                history.append("->").append(line);
                results.add("{\"key\\":[\"" + history.toString() + "\\"]}");
            } else {
                results.add("{\"key\\":[]}");
            }
        }

        scanner.close();

        System.out.print("[");
        for (int i = 0; i < results.size(); i++) {
            System.out.print(results.get(i));
            if (i < results.size() - 1) System.out.print(",");
        }
        System.out.println("]");
    }
}
```

Problem 5: Guarded Transitions: Error Handling

Description: Create a Finite State Machine that incorporates guarded transitions with error handling. The FSM should have states such as 'IDLE' and 'ACTIVE'. Certain events ('RUN') should trigger valid transitions, while others ('FLY') should be considered invalid, leading to an 'ERROR' state or appropriate error indication.

```
import java.util.*;

public class Solution {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        List<String> results = new ArrayList<>();
        String state = "IDLE";

        while (scanner.hasNextLine()) {
            String line = scanner.nextLine().trim();

            if (!state.equals("ERROR")) {
                if (state.equals("IDLE") && line.equals("RUN")) {
                    state = "ACTIVE";
                }
                else if (state.equals("ACTIVE") && line.equals("STOP")) {
                    state = "IDLE";
                }
                else {
                    state = "ERROR";
                }
            }

            results.add("{\"key\": \"" + state + "\"}");
        }

        scanner.close();

        System.out.print("[");
        for (int i = 0; i < results.size(); i++) {
            System.out.print(results.get(i));
            if (i < results.size() - 1) System.out.print(",");
        }
        System.out.println("]");
    }
}
```

8. Learning Outcomes

- Gained a practical understanding of Finite State Machine (FSM) theory and its application in software design.
- Developed proficiency in implementing state management and event-driven transitions in Java.
- Acquired experience in modeling diverse system behaviors using FSMs, from simple toggles to complex cycles and error handling.
- Enhanced problem-solving skills by breaking down complex requirements into manageable state-based logic.

9. Challenges Faced & Solutions

Challenge: Managing complex state transitions and ensuring all possible input events were handled correctly. Solution: Utilized a clear, object-oriented approach where each state was represented by a distinct object or enumeration, and transition logic was explicitly defined within the FSM core or state objects, supported by thorough testing.

Challenge: Implementing the state history tracking without introducing significant overhead. Solution: Employed a `List` or `ArrayList` to store the sequence of visited states, appending each new state efficiently upon transition, and ensuring the history was accessible for reporting.

10. Conclusion

The successful implementation of various Finite State Machine models demonstrates the power and flexibility of this computational model in software development. By addressing problems ranging from simple toggles to complex state cycles and error handling, this project underscores the FSM's capability to manage intricate system behaviors effectively and maintainable.

11. Future Scope

- Extend the FSM to support hierarchical state machines for managing more complex nested states.
- Implement a graphical visualization tool to represent the FSM structure and transitions dynamically.
- Integrate the FSM with external event sources or user interfaces for real-time interaction and control.

12. References

- Java Documentation: The official Oracle documentation for the Java programming language.
- "Introduction to Automata Theory, Languages, and Computation" by John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman: A foundational textbook on theoretical computer science, including FSMs.
- Online tutorials and articles on Finite State Machine design patterns in Java.

13. Annexure

- Internship Completion Certificate
- Offer Letter